

MEDORA

Project Folder Structure: Frontend and Backend

AI Medical Memory and Care Companion | Understand Your Medical Story. Navigate Your Care.

1. Overview and Recommended Stack

This document lays out how the MEDORA codebase should be organized so that a four-person team can work in parallel without stepping on each other. The structure follows the AI pipeline in the project document: **Upload, OCR, Extraction, Medical Memory, Retrieval, RAG, Simplified Answer**. Each pipeline stage has its own folder in the backend, and each prototype feature has its own components, page and service in the frontend.

Layer	Free technology	Cost
Frontend	React + Vite + Tailwind CSS, React Router, Axios	Free, open source
Backend	Python FastAPI + Pydantic + Uvicorn	Free, open source
Database	SQLite (one local file) + SQLAlchemy	Free, nothing to install
OCR / PDF text	Tesseract (pytesseract) + pdfplumber	Free, runs locally
Embeddings	sentence-transformers (local model)	Free, no API needed
Vector store	ChromaDB (local folder)	Free, runs locally
Main LLM	Google Gemini API, free tier (Flash model)	Free, no card; limits vary
Backup LLM	Ollama with a local model (for example llama3.2)	Free, unlimited, offline
Demo / hosting	Localhost on a laptop (no hosting needed)	Free

Everything above is free of cost: no paid API, no cloud service and no card is needed. Removed from the earlier version: cloud OCR, PostgreSQL server and Docker files. Every tool sits behind one service file, so any of them can be swapped without touching the rest of the code.

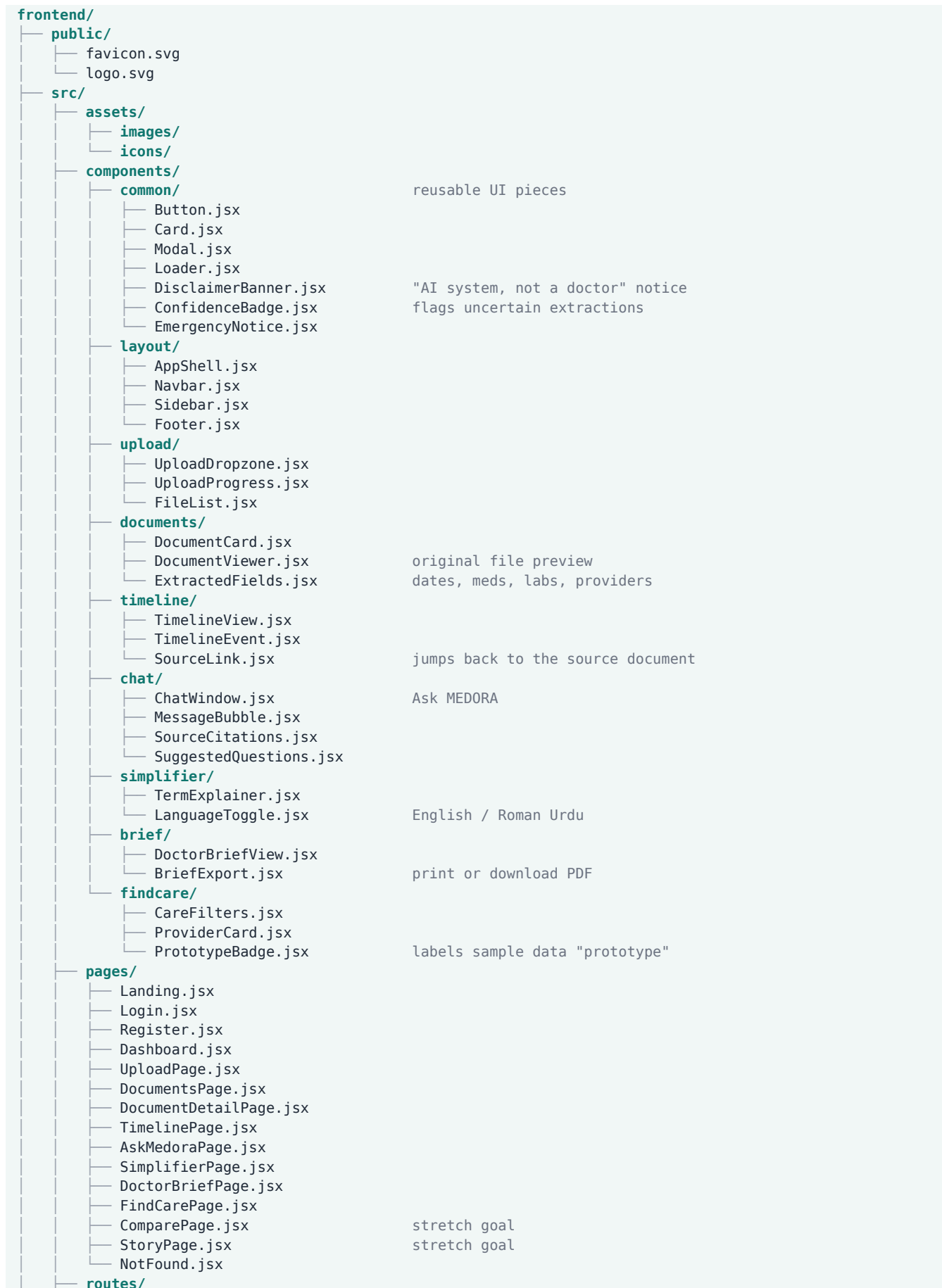
2. Root Repository (Monorepo)

One Git repository with two clearly separated apps, plus shared documentation and demo data.

medora/	
├─ frontend/	React web app (see section 3)
├─ backend/	FastAPI server, AI pipeline (see section 4)
├─ docs/	
│ └─ MEDORA_Final_Hackathon_Document.docx	project document
│ └─ architecture.md	pipeline diagram and decisions
│ └─ api-spec.md	endpoint list shared by both teams
│ └─ demo-script.md	step-by-step live demo flow
│ └─ responsible-ai.md	safety and disclaimer rules
├─ sample-data/	
│ └─ reports/	demo PDF and one scanned image (no real patient data)
│ └─ expected-output/	what extraction should produce, for testing
├─ .gitignore	node_modules, .env, storage/, vector_index/
├─ .env.example	shared variable names, no secrets, no keys
└─ README.md	how to install and run in 5 minutes

3. Frontend Folder Structure

Rule of thumb: pages assemble the screen, components draw the pieces, services talk to the backend, hooks hold the logic. A component never calls the API directly; it always goes through a service.



```
├── AppRoutes.jsx
├── ProtectedRoute.jsx
├── services/
│   ├── api.js
│   ├── authService.js
│   ├── documentService.js
│   ├── timelineService.js
│   ├── chatService.js
│   ├── simplifierService.js
│   ├── briefService.js
│   └── careService.js
├── hooks/
│   ├── useAuth.js
│   ├── useUpload.js
│   ├── useDocuments.js
│   ├── useTimeline.js
│   ├── useChat.js
│   └── useLanguage.js
├── context/
│   ├── AuthContext.jsx
│   └── LanguageContext.jsx
├── utils/
│   ├── formatDate.js
│   ├── validators.js
│   └── constants.js
├── styles/
│   └── index.css
├── App.jsx
├── main.jsx
├── .env.example
├── index.html
├── package.json
├── tailwind.config.js
├── postcss.config.js
├── vite.config.js
└── README.md
```

blocks pages when not signed in
all backend calls live here
Axios instance, base URL, auth header

file type and size checks

Tailwind directives

VITE_API_URL=

4. Backend Folder Structure

Rule of thumb: endpoints stay thin; the real work happens in services/. The services folder is organized in the same order as the AI pipeline, so anyone can find the stage they need.

backend/	
app/	
main.py	creates the FastAPI app, CORS, routers
core/	
config.py	reads .env settings
security.py	password hashing, JWT tokens
database.py	SQLite engine and session (free)
logging.py	app logs and audit logging setup
api/	
deps.py	current user, DB session dependencies
v1/	
router.py	joins all endpoint routers
endpoints/	
auth.py	register, login, me
documents.py	upload, list, get, delete
timeline.py	events with source links
chat.py	Personal RAG question answering
simplifier.py	simple English / Roman Urdu
brief.py	Doctor Brief generation
care.py	Find Care sample providers
compare.py	stretch goal
models/	database tables
user.py	
document.py	
extracted_item.py	medication, lab, procedure, provider...
medical_event.py	timeline entries
chat_session.py	
audit_log.py	who viewed or shared what
schemas/	request and response shapes (Pydantic)
auth.py	
document.py	
timeline.py	
chat.py	
brief.py	
care.py	
services/	
ingestion/	stage 1-2: upload, OCR, text
file_validator.py	type, size, safety check
ocr_service.py	Tesseract, runs locally
text_extractor.py	PDF text layer or OCR text
document_classifier.py	lab report, prescription, ...
extraction/	stage 3: structured details
medical_extractor.py	
confidence_scorer.py	flags low-confidence values
memory/	stage 4: Medical Memory
memory_builder.py	joins data across documents
timeline_builder.py	
rag/	stage 5-6: retrieval and answers
chunker.py	
embedder.py	sentence-transformers, local
vector_store.py	ChromaDB, local folder
retriever.py	only this patient's records
rag_pipeline.py	retrieve, prompt, answer, cite
llm/	
llm_client.py	single place that calls the LLM
gemini_provider.py	Gemini free-tier API
ollama_provider.py	local backup, no limits
prompts/	
extraction.txt	
qa_answer.txt	
simplify.txt	
doctor_brief.txt	
simplifier/	
simplifier_service.py	
fact_guard.py	checks doses, dates, values unchanged
brief/	

- brief_generator.py - care/ - care_service.py - safety/ - disclaimer.py - emergency_detector.py - response_guard.py - utils/ - file_helpers.py - date_parser.py - data/ - sample_providers.json - storage/ - medora.db - uploads/ - vector_index/ - scripts/ - seed_demo_data.py - tests/ - test_ingestion.py - test_extraction.py - test_rag.py - test_fact_guard.py - test_safety.py .env.example - requirements.txt - README.md	- filters the sample provider list - Responsible AI rules in code - points to emergency services - blocks diagnosis or prescriptions - hand-made, labeled "prototype" - git-ignored - SQLite database file - user documents, private - embeddings - loads demo patient for the live demo - LLM_PROVIDER, GEMINI_API_KEY, GEMINI_MODEL, OLLAMA_MODEL, JWT_SECRET - all free packages
---	--

5. Feature-to-Folder Map

Use this table to see exactly which files belong to each prototype feature. It also makes it easy to split the work between teammates.

Feature	Frontend files	Backend files
Document upload and OCR	components/upload, UploadPage, documentService, useUpload	endpoints/documents.py, services/ingestion/*
Information extraction	components/documents, DocumentDetailPage	services/extraction/*, models/extracted_item.py
Medical timeline	components/timeline, TimelinePage, timelineService	endpoints/timeline.py, services/memory/*
Personal RAG Q&A	components/chat, AskMedoraPage, chatService, useChat	endpoints/chat.py, services/rag/*, services/llm/*
Language simplifier	components/simplifier, SimplifierPage, LanguageContext	endpoints/simplifier.py, services/simplifier/*
Doctor Brief	components/brief, DoctorBriefPage, briefService	endpoints/brief.py, services/brief/*
Find Care (prototype)	components/findcare, FindCarePage, careService	endpoints/care.py, services/care/*, data/sample_providers.json
Responsible AI and safety	DisclaimerBanner, EmergencyNotice, ConfidenceBadge	services/safety/*, models/audit_log.py
Sign-in and privacy	AuthContext, ProtectedRoute, Login, Register	core/security.py, endpoints/auth.py, api/deps.py

6. Main API Endpoints

Method	Endpoint	Purpose
POST	/api/v1/auth/register and /login	Create account, sign in, get token
POST	/api/v1/documents/upload	Upload PDF or image; starts OCR and extraction
GET	/api/v1/documents	List the user's documents
GET	/api/v1/documents/{id}	Document details and extracted fields

Method	Endpoint	Purpose
DELETE	/api/v1/documents/{id}	Safe deletion of a document and its data
GET	/api/v1/timeline	Ordered medical events with source links
POST	/api/v1/chat/ask	Question in, source-grounded answer out
POST	/api/v1/simplify	Explain a term or text in simple English or Roman Urdu
POST	/api/v1/brief/generate	Create the Doctor Brief
GET	/api/v1/care/search	Filter sample providers (specialty, location, cost, mode)

7. Conventions the Team Should Follow

Area	Rule
Naming	React components and pages: PascalCase .jsx. Hooks: useSomething .js. Python files and folders: snake_case . Endpoints: plural nouns, lowercase.
Separation	Components never call the API directly (use services/). Endpoints never contain AI logic (use services/). Only <code>llm_client.py</code> talks to the LLM (Gemini free key, or Ollama as backup).
Safety	Every AI answer passes through <code>response_guard.py</code> and carries the disclaimer. Low-confidence values are always flagged.
Secrets	Real keys live only in <code>.env</code> , never in Git. Commit only <code>.env.example</code> .
Patient data	Use only fake demo data in <code>sample-data/</code> (free Gemini tier may use prompts for training). The storage/ folder is git-ignored. Retrieval is always filtered by the signed-in user's ID.
Git workflow	Branches: <code>main</code> (stable demo), <code>dev</code> , and <code>feature/<name></code> . Small commits, pull request before merging into <code>dev</code> .
Stretch goals	Compare and Story files can stay as empty placeholders until the core demo is solid.

8. Free Setup Checklist

Step	What to do	Cost
1	Get a Gemini key at aistudio.google.com (Google account, Get API key). Put it in <code>backend/.env</code> as <code>GEMINI_API_KEY</code> .	Free
2	Install Tesseract, then: <code>pip install pytesseract pdfplumber</code>	Free
3	<code>pip install fastapi uvicorn sqlalchemy python-dotenv httpx sentence-transformers chromadb</code>	Free
4	Install Ollama (ollama.com) and run: <code>ollama pull llama3.2</code> . Set <code>LLM_PROVIDER=ollama</code> if Gemini hits its limit.	Free
5	Frontend: <code>npm create vite@latest</code> , then <code>npm install axios react-router-dom tailwindcss @tailwindcss/vite</code>	Free
6	Demo from localhost. Use only fake reports, and save every result in SQLite so each document is processed once.	Free

Gemini free-tier limits are shown per project in Google AI Studio and can change, so check them before the demo.

MEDORA is a tool for understanding information and finding care. It does not replace a doctor.